MANIPAL UNIVERSITY JAIPUR

School of Computer Science & Engineering

Department of Computer Science & Engineering

PROJECT SYNOPSIS

for Android App Development Laboratory

Project Title

“Pocketly: Personal Finance & Expense Tracker”

Student Name	**Aryan Rastogi**
Registration No.	**2427030083**
Course / Lab	Android App Development Laboratory (CS3102)
Section	Android App Dev Section C
Department	Computer Science & Engineering
Institution	Manipal University Jaipur, Rajasthan
Academic Session	2026 – 2027
Submission Track	Laboratory Project Synopsis & Architecture Report

Declaration: This synopsis presents original work carried out for the Android App Development Laboratory at Manipal University Jaipur. The application has been fully implemented natively using Kotlin, Jetpack Compose, and SQLite on Android Studio.

SYNOPSIS — SUBMISSION GUIDELINES

1. Project Title & Problem Statement

Project Title: Pocketly: Personal Finance & Expense Tracker

Problem Statement: In contemporary digital economies, individuals—specifically college students, young working professionals, and freelancers—face acute friction when attempting to track daily expenditures and monitor personal cash reserves. The ubiquitous penetration of instant contactless payment ecosystems, such as UPI (Unified Payments Interface), credit cards, and digital wallets, has drastically reduced the friction of spending money, but inversely increased the difficulty of conscious money management. Users frequently experience end-of-month budget shocks because small daily expenses (such as meals, local transit fares, and digital subscriptions) go unmonitored.

Conventional personal finance and expense tracking applications suffer from five critical limitations:

· **High Logging Friction:** Overly complex forms and multi-nested navigation menus discourage regular entry of frequent micro-transactions.

· **Privacy Vulnerabilities & Cloud Telemetry:** Mainstream commercial applications aggressively harvest personal banking SMS data, contact books, and transaction records to remote ad-brokers without explicit user consent.

· **Lack of Real-Time Visual Feedback:** Traditional apps display dry tabular numbers without contextual visual cues, rendering it difficult to perceive spending velocity or category ratios at a glance.

· **Strict Cloud Dependency:** Most commercial apps refuse to function in offline environments such as basements, metro commutes, flights, or rural regions.

· **Ad Bloat & Inappropriate Upselling:** Free-tier apps degrade user trust by injecting aggressive pop-ups, loan offerings, and third-party sponsored content.

2. Objectives & Proposed Solution

Main Objectives:

· **To analyze functional requirements:** Identify essential touchpoints in daily personal expense management and translate them into a low-latency native mobile workflow.

· **To design a high-fidelity user interface:** Implement a bespoke *Liquid Glass* design language using Jetpack Compose and Material 3, emphasizing visual hierarchy, specular translucency, and elegant typography.

· **To build an offline-first architecture:** Utilize an ACID-compliant local SQLite database (via PocketlyDbHelper) ensuring zero network latency, full offline availability, and absolute data privacy.

· **To deliver dynamic Canvas-drawn analytics:** Construct hardware-accelerated segmented donut charts and monthly spending trend bar charts directly rendered via Jetpack Compose Canvas.

· **To support multi-account liquidity:** Enable seamless tracking across multiple bank accounts (e.g., HDFC Bank, SBI) with aggregated available balances and institutional tagging.

· **To demonstrate core Android competencies:** Implement Activity lifecycle handling, unidirectional state hoisting (remember, mutableStateOf), form validation, and local key-value caching (SharedPreferences).

Proposed Solution: Pocketly provides an elegant, distraction-free, and privacy-centric personal finance companion. Pocketly completely eliminates expense logging friction through a persistent quick-action Floating Action Button, categorized quick-entry modals, and instant recalculation of available liquid balances. By retaining 100% of data within an encrypted local SQLite database, Pocketly guarantees absolute user privacy without requiring third-party cloud accounts or network connectivity. Interactive visual feedback—featuring donut expense rings, safe daily spend guidance, and monthly comparative bar charts—empowers users to maintain financial discipline effortlessly.

3. App Components / Features

Pocketly is engineered with a modular, scalable architecture combining reactive declarative user interface components, unidirectional state hoisting, and an offline-first relational persistence engine. The application comprises seven major modules:

· **Authentication & Session Management (LoginScreen):** Frosted Liquid Glass authentication portal featuring dual tabs (Sign In / Create Account), email format and password length validation, interactive password visibility toggle, 'Remember Me' preference persistence, and smooth crossfade transitions.

· **Dashboard & Overview Feed (HomeScreen):** The primary command center orchestrating the serif TopBar, contextual GreetingSection, BalanceCard (Available Balance, Spent, and Received metrics with directional visual badges), ConnectedAccountsCard carousel, and MonthlySpendingCard.

· **Visual Spending Breakdown & Canvas Donut Ring (MonthlySpendingCard & DonutChart):** Hardware-accelerated custom Canvas drawing engine rendering proportional circular arcs for expense categories (Food, Mobile Bills, Shopping, Transport), percentage ratios, spend reduction delta (down 23% vs. last month), and an interactive month picker.

· **Transaction Ledger & Dynamic Filter Engine (TransactionsScreen):** Real-time transaction feed featuring category chip filters ('All', 'Expenses', 'Income'), chronologically grouped items, and instant search query matching directly against SQLite database records.

· **Add Transaction Form Dialog (AddTransactionDialog):** Low-friction modal dialog supporting dual transaction types (Expense vs. Income) via RadioButtons, payment method chips (UPI, Cash, Cheque), category dropdown spinner, amount field, and recurring monthly CheckBox.

· **Transaction Receipt & Digital Invoice Modal (TransactionDetailDialog):** Interactive receipt dialog displaying merchant avatar, payment verification pill ('Paid Successfully'), transaction timestamp, debit account reference ID, and action triggers for receipt sharing and invoice export.

· **Budget Management & Analytics (AnalyticsScreen):** Visual analytics hub featuring a monthly target dial (Rs. 15,000 budget), safe daily spend calculation (Rs. 500 / day), circular budget consumption progress ring (43% used), and interactive monthly spending trend bar charts.

· **User Profile & Security Controls (ProfileScreen):** User credential overview (Aryan Rastogi, aryanfeb17@gmail.com), Primary UPI VPA card with one-tap clipboard copy feedback, push notification toggles, budget exceeded alert switches, and secure sign-out dialog.

Android System Components & Architecture

The project demonstrates key Android development concepts: a Single-Activity architecture hosted in MainActivity (extending ComponentActivity) with edge-to-edge transparent system bars; declarative UI construction using Jetpack Compose; state management via remember, mutableStateOf, and unidirectional state hoisting; custom Compose Canvas rendering (drawArc, drawRoundRect, Path); and smooth momentum trackpad physics via SmoothScrollContainer and rememberFluidFlingBehavior.

Database Schema & Local Persistence Engine

Data persistence is managed via an optimized SQLite database implementation in PocketlyDbHelper. The relational schema maintains ACID guarantees and executes parameterized SQL queries for atomic insertions, updates, and aggregate queries. User credentials and session tokens are preserved via Android SharedPreferences (UserPreferences).

Table	Columns	Data Types	Functional Role
transactions	id, title, amount, category, type, date, account, note	INTEGER PK, TEXT, REAL, TEXT, TEXT, INTEGER, TEXT, TEXT	Stores individual credit/debit records with categorization
accounts	id, bank_name, account_number, balance, is_primary	INTEGER PK, TEXT, TEXT, REAL, INTEGER	Maintains connected banking accounts and balances
preferences	user_name, user_email, is_logged_in, remember_me	SharedPreferences Key-Value Store	Preserves authentication state and security preferences

4. App Screenshots (Visual Showcase of Working Application)

The figures below present authentic, high-resolution screen captures recorded directly from the running Pocketly application on an Android Virtual Device (Google Pixel 8 running Android 15, API Level 35). They illustrate the refined Liquid Glass design system, dynamic Compose animations, and real-time SQLite database reactivity.

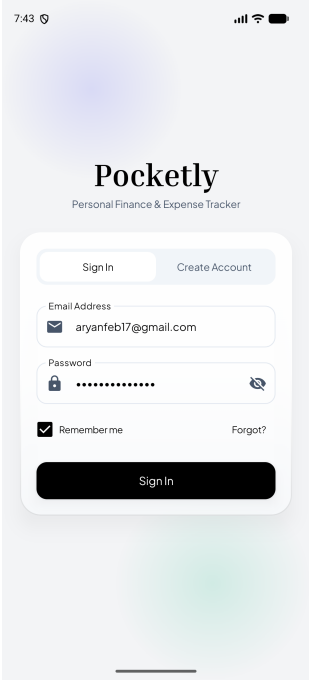


Figure 1: Liquid Glass Login & Authentication Screen
Client-side email/password validation, Remember Me checkbox, and frosted glass card elevation.

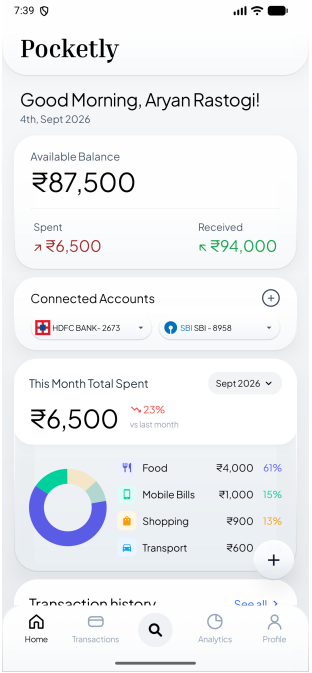


Figure 2: Home Dashboard & Spending Overview
Live Available Balance card, Connected Bank Accounts pill carousel, and Canvas Donut Chart.

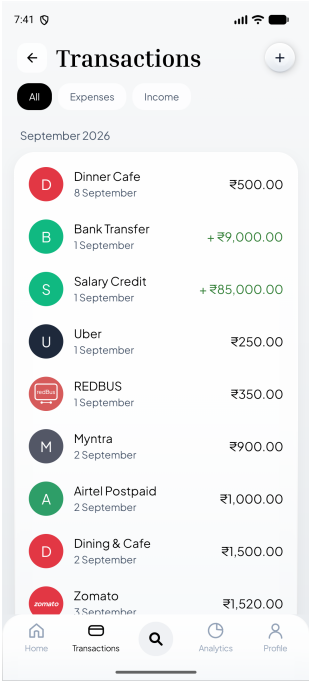


Figure 3: Filterable Transaction History Ledger
Category chips (All/Expenses/Income), chronological date sorting, and instant SQLite query updates.

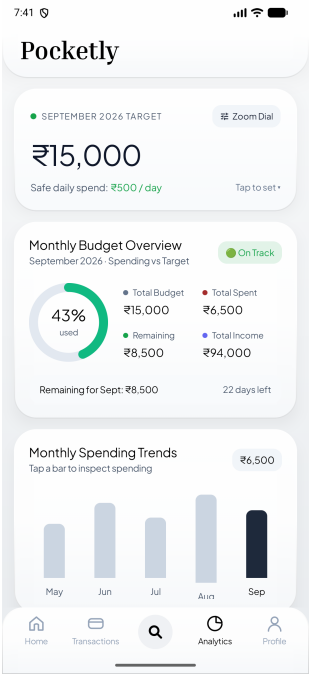


Figure 4: Spending Analytics & Budget Management
Monthly budget target dial, 43% utilization progress ring, safe daily spend, and interactive bar charts.

4. App Screenshots (Interactive Modals & System Settings)

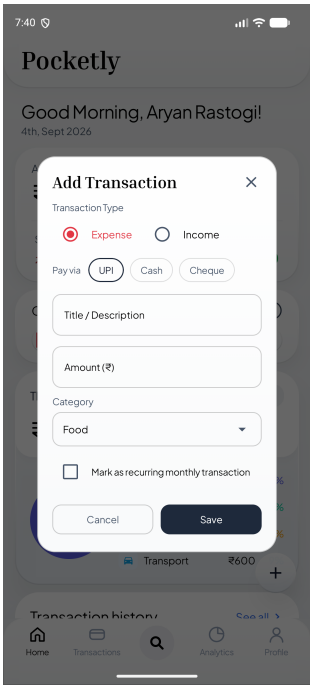


Figure 5: Add Transaction Form Modal
RadioButtons for type, payment mode chips (UPI/Cash/Cheque), Category Spinner, and recurring CheckBox.

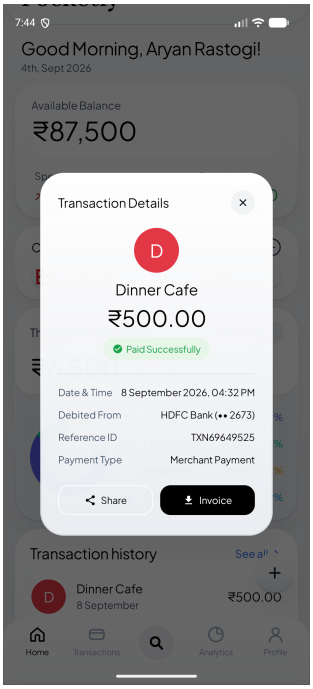


Figure 6: Digital Transaction Receipt Dialog
Payment verification badge ('Paid Successfully'), bank reference ID, and Share/Invoice actions.

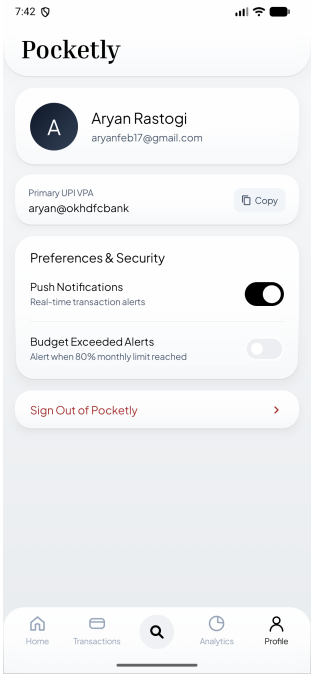


Figure 7: User Profile & Security Controls
User avatar, UPI VPA clipboard copy, push & budget alert toggles, and sign-out dialog.

Key UI/UX Innovations Demonstrated:

· **Liquid Glass Design System:** Custom composables (GlassCard, GlassPill) with vertical specular gradients, translucent elevation, and soft blur simulation.

· **Editorial Typography:** Dual Google font integration pairing *Inria Serif* for display branding and *Plus Jakarta Sans* for high-legibility numerals and labels.

· **Hardware-Accelerated Canvas Graphics:** Custom mathematical arc calculations for donut segments and interactive progress dials without external charting libraries.

· **Offline Privacy by Design:** Zero telemetry or external network calls; complete local data sovereignty via SQLite and SharedPreferences.

· **Fluid Trackpad & Touch Dynamics:** Fling behavior interception engineered for high-refresh-rate physical devices and desktop development emulators.

Architecture Summary: High-performance Compose components adhering to strict Material 3 design tokens.

5. Usability & Real-World Impact

Target Users: Pocketly is tailored for college students managing monthly pocket money or stipends, young working professionals navigating recurring salaries and utility obligations, freelancers tracking irregular multi-account inflows, and privacy-conscious smartphone users who demand confidential expense tracking without cloud surveillance.
Practical Use Cases: On-the-go logging of quick daily retail, transit, and food purchases; immediate verification of net available liquidity across multiple banks; monitoring remaining monthly budget quotas via safe daily spend guidance; and retrieving instant digital receipts for expense reimbursements.
Real-World Benefits: Cultivates financial mindfulness through instant visual feedback, eliminates debt traps through early budget warnings, operates 100% offline without cellular data dependency, and ensures complete zero-knowledge data privacy.

6. Technology Used

Component / Layer	Technology / Tool	Version / Specification	Purpose & Implementation Detail
Programming Language	Kotlin	v1.9.24+	Modern, type-safe application logic & coroutines
Development IDE	Android Studio	Ladybug (2024.2+) / Iguana	Official Android IDE, layout inspector & build tools
UI Framework	Jetpack Compose	Compose BOM 2024.09.00	Declarative reactive UI, Material 3 design tokens
Local Relational DB	SQLite / SQLiteOpenHelper	Android SQLite Core	ACID transactions, indexed tables, aggregate queries
Key-Value Storage	SharedPreferences	AndroidX Core KTX	UserPreferences session state, Remember Me, settings
Graphic Engine	Compose Canvas API	Hardware-accelerated	Custom segmented donut chart & budget progress rings
Typography	Google Fonts	Inria Serif & Plus Jakarta Sans	High-contrast branding and high-legibility UI numerals
Build Toolchain	Gradle (Kotlin DSL)	Gradle 8.7 / AGP 8.5+	Automated dependency resolution & APK compilation
Platform Targets	Android OS	Min SDK 26 / Target SDK 35	Android 8.0 Oreo through Android 15 Vanilla Ice Cream

7. Future Scope & Planned Enhancements

· **Automated SMS Transaction Ingestion:** Integration of the Android SMS Retriever API or BroadcastReceiver to automatically parse banking SMS alerts and record expenses with zero manual input.

· **Biometric Authentication:** Integration of AndroidX BiometricPrompt to enable fingerprint and facial recognition unlocking for heightened financial security.

· **Financial Statement Export:** Generation of monthly PDF statements and CSV exports for tax reporting, personal accounting, and spreadsheet analysis.

· **Encrypted Cloud Backup:** Optional zero-knowledge peer-to-peer or encrypted cloud synchronization (via Google Drive or Supabase) with user-controlled keys.

· **Intelligent Spending Alerts:** Android WorkManager background tasks triggering proactive push notifications whenever daily spending velocity threatens monthly budget limits.

· **On-Device Receipt OCR Scanning:** Google ML Kit Text Recognition integration allowing users to snap photos of physical store receipts to automatically populate merchant, date, and amount fields.

8. Student Verification & Evaluation Sign-off

I hereby affirm that the project titled **“Pocketly: Personal Finance & Expense Tracker”** represents original work developed under the Android App Development Laboratory curriculum. All components, layouts, SQLite database functions, and screenshots presented herein reflect the functioning Android application.

Student Name:	**Aryan Rastogi**	**Faculty Guide Signature:**	_____
Registration No.:	**2427030083**	**Evaluation Date:**	October 2026
Section:	Android App Dev Section C	**Student Signature:**	**Aryan Rastogi**